



UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE FOR ROBOTICS II

Final Assignment

Domestic Service Robot – Heating Dynamics

Author:

Crovara Matteo

Student ID:

s5199657

Professor:

Mastrogiovanni Fulvio

Kashmar Omar

June 12, 2026

Contents

1	Introduction	2
2	The Discrete PDDL Model	2
2.1	Environment and State Representation	2
2.2	Action Space and Robotic Manipulation	2
2.3	Modeling Discrete Thermodynamics	3
2.4	Test Case 1: Immediate Heating	3
2.5	Test Case 2: The Importance of Ordering	4
3	The Continuous PDDL+ Model	4
3.1	Continuous Time and Differential Processes	4
3.2	Autonomous Safety Events	5
3.3	The Robot Cooldown Mechanism	5
3.4	Test Case 3: Autonomous Safety and Thermal Tracking	5
4	Reflection and Limitations	6
4.1	Limitations and Planner Behavior	6

1 Introduction

The purpose of this work is to model the automated planning domain of a domestic service robot tasked with the preparation of hot beverages. The considered scenario integrates concepts from autonomous task planning, resource management, and thermodynamics, with the goal of representing both the logical sequence of manipulation actions and the physical evolution of the environment over time.

The domain is designed to describe a robotic agent capable of interacting with standard kitchen appliances, a water dispenser and a kettle to prepare various types of drinks, such as flavored water, warm tea, and hot infusions. Each beverage requires distinct and strict thermal conditions. Particular attention is given to the progression between two different planning paradigms: an initial discrete model and a more advanced continuous one.

In the first part of the project (Q1), the domain is formalized using standard numeric PDDL. Temperature and water volume are modeled as discrete numeric fluents, and physical processes like water heating are abstracted as step-by-step actions. This foundational model serves to validate the core logic of the robot's manipulation capabilities and the correct ordering of tasks.

In the second part (Q2), the model is significantly extended into the PDDL+ paradigm. This upgrade introduces continuous time ($\#t$), differential equations for heating and cooling processes, and autonomous safety events. This transition bridges the gap between abstract logical planning and real-world physical simulation, forcing the robotic agent to account for temporal inertia, thermal dissipation, and environmental constraints.

The consistency, temporal validity, and logical soundness of both models are verified using the ENHSP. This specific solver was selected due to its robust support for numeric fluents and continuous time dynamics, ensuring that the generated plans are mathematically accurate and physically feasible.

2 The Discrete PDDL Model

The first phase of the project focused on developing a purely discrete numeric model to validate the sequential logic and spatial constraints of the robotic agent. In this paradigm, physical phenomena such as thermodynamics and fluid transfer are abstracted into instantaneous, step-by-step actions.

2.1 Environment and State Representation

The modeled environment represents a domestic kitchen equipped with standard appliances and resources. The domain features stationary structural elements, specifically a `heater base` and a `water dispenser`, mobile containers such as `kettles` and `cups`, and various ingredients (`flavor-drop`, `tea-bag`, and `herbal-bag`).

The primary objective of the single-armed robotic agent is to manipulate these objects to prepare specific beverages. A core feature of the domain is that each drink imposes strict thermal preconditions for its preparation:

- **Flavored Water:** requires cold water, specifically $T \leq 30$ °C.
- **Warm Tea:** requires a medium-high temperature, restricted to $55 < T \leq 65$ °C.
- **Hot Infusion:** requires near-boiling water, operating in the $85 < T \leq 95$ °C range.

To track the physical properties of the liquids, the model relies on specific numeric fluents: `(temperature ?k)`, `(temperature-cup ?c)`, and `(water-volume ?k)`. The introduction of fresh water is managed by the `fill-kettle` action: whenever the kettle is filled at the dispenser, the influx of new liquid automatically resets the kettle's thermal state to an ambient baseline of 20°C. Subsequently, the thermal transfer between the kettle and the cup is handled efficiently by the `pour-water` action. When executed, the robot transfers exactly 250ml of liquid to an empty cup and simultaneously assigns the kettle's current temperature to the cup's thermal fluent. This interplay of numeric assignments allows the final preparation actions to rigorously validate the required temperature ranges before the ingredient is inserted.

2.2 Action Space and Robotic Manipulation

The operational capabilities of the robotic agent are defined by a comprehensive set of instantaneous actions, which can be logically grouped into three main functional categories:

1. Spatial Manipulation (Pick and Place): Because the robot is modeled as a single-armed agent, all grasping actions strictly require the `(handempty)` precondition. The domain explicitly maps the kinematic transitions between the different operational zones: the table, the heater, and the dispenser. Pairs of inverse actions, such as `pick-from-heater` and `place-on-heater`, carefully manage the topological state of the objects. Crucially, placing a kettle on an appliance requires the appliance to be empty (e.g., `dispenser-empty ?d`), ensuring physical non-compensation.

2. Appliance Interaction: The robot acts as an external trigger for the kitchen appliances. Actions like `turn-on-heater` and `turn-off-heater` simply toggle boolean predicates (`heater-on`), which in turn enable specific thermal effects. Similarly, the `fill-kettle` action represents the interaction with the water supply; it specifically requires the kettle to be physically located on the dispenser (`on-dispenser ?k ?d`) and increases the numeric volume variable by a fixed amount of 250ml per execution, up to a maximum capacity of 750ml.

3. Beverage Assembly: This category encompasses the high-level tasks that fulfill the domain's goals. It begins with the `pour-water` action, which links the volumetric decrease in the kettle with the state change of the cup (`not (cup-empty ?c)`). Following the pouring, the robot can execute the `pick-ingredient` action. The final step is handled by specialized preparation actions (`prepare-flavored-water`, `prepare-warm-tea`, `prepare-hot-infusion`). These actions act as strict logical gates: they consume the ingredient held by the robot and lock the beverage state into "ready", but only if the numeric validation of the cup's temperature strictly satisfies the mathematical boundaries defined for that specific drink.

2.3 Modeling Discrete Thermodynamics

In the absence of continuous time derivatives, the heating process was modeled through an iterative action called `heat-step`. When the kettle is placed on the active heater, the robot can execute this action to instantaneously increase the water temperature by a fixed quantum (10°C per step):

```
(increase (temperature ?k) 10)
```

This granular approach requires the planner to calculate the exact number of heating iterations required to reach the target thermal window (e.g., exactly 4 steps to reach 60°C from an initial temperature of 20°C, perfectly satisfying the requirement for warm tea).

2.4 Test Case 1: Immediate Heating

To verify the basic capabilities of the model, we designed a foundational problem where the robot must prepare two identical warm teas. The environment provides all necessary appliances and resources.

```
0.0: (pick-from-table k1)
1.0: (place-on-dispenser k1 d1)
2.0: (fill-kettle k1 d1)
3.0: (fill-kettle k1 d1)
4.0: (fill-kettle k1 d1)
5.0: (pick-from-dispenser k1 d1)
6.0: (place-on-heater k1 h1)
7.0: (turn-on-heater h1 k1)
8.0: (heat-step k1 h1)
9.0: (heat-step k1 h1)
10.0: (heat-step k1 h1)
11.0: (heat-step k1 h1)
12.0: (turn-off-heater h1 k1)
13.0: (pick-from-heater k1 h1)
14.0: (pour-water k1 cup1)
15.0: (pour-water k1 cup2)
16.0: (place-on-table k1)
17.0: (pick-ingredient tea1)
18.0: (prepare-warm-tea cup2 tea1)
19.0: (pick-ingredient tea2)
20.0: (prepare-warm-tea cup1 tea2)
```

The solver successfully generated a linear plan demonstrating the correct abstraction of the physical processes:

- **Resource Gathering (0.0 - 5.0):** The robot picks the kettle, places it on the dispenser, and executes three consecutive `fill-kettle` actions to gather 750ml of water at 20°C.
- **Immediate Heating (6.0 - 12.0):** The kettle is moved to the heater and turned on. The planner executes exactly four `heat-step` actions, bringing the numeric fluent `(temperature ?k)` to 60°C.
- **Execution (13.0 - 20.0):** The heater is turned off, and the robot pours the 60°C water into both cups before sequentially picking and inserting the tea bags.

2.5 Test Case 2: The Importance of Ordering

A more complex scenario was engineered to demonstrate how strict ordering matters in a discrete numeric environment. In this problem, the robot must prepare a *warm tea* ($55 < T \leq 65^\circ\text{C}$) and a *hot infusion* ($85 < T \leq 95^\circ\text{C}$).

To force the planner into a strict sequence, we intentionally removed the `dispenser` from the initial state and provided a kettle already filled with exactly 500ml of cold water. Because the discrete model lacks a cooling process, any excess heat cannot be dissipated over time. If the robot mistakenly heats the entire volume of water to 90°C for the infusion first, it will reach a thermal dead-end, as the remaining water cannot be cooled back down to 60°C for the tea.

```

0.0: (pick-from-table k1)
1.0: (place-on-heater k1 h1)
2.0: (turn-on-heater h1 k1)
3.0: (heat-step k1 h1)
4.0: (heat-step k1 h1)
5.0: (heat-step k1 h1)
6.0: (heat-step k1 h1)
7.0: (turn-off-heater h1 k1)
8.0: (pick-from-heater k1 h1)
9.0: (pour-water k1 cup1)
10.0: (place-on-table k1)
11.0: (pick-ingredient teal)
12.0: (prepare-warm-tea cup1 teal)
13.0: (pick-from-table k1)
14.0: (place-on-heater k1 h1)
15.0: (turn-on-heater h1 k1)
16.0: (heat-step k1 h1)
17.0: (heat-step k1 h1)
18.0: (heat-step k1 h1)
19.0: (turn-off-heater h1 k1)
20.0: (pick-from-heater k1 h1)
21.0: (pour-water k1 cup2)
22.0: (place-on-table k1)
23.0: (pick-ingredient herbal1)
24.0: (prepare-hot-infusion cup2 herbal1)

```

The solver successfully navigated this logical trap by generating a strictly ordered plan:

- **Phase 1 (0.0 - 12.0):** The robot first heats the water to 60°C (using 4 `heat-step` actions) and pours 250ml to satisfy the warm tea precondition.
- **Phase 2 (13.0 - 24.0):** *Only after* securing the lower-temperature beverage, it places the kettle back on the heater. It executes 3 additional `heat-step` actions to bring the remaining 250ml of water from 60°C to 90°C , finally pouring it for the hot infusion.

This test case proves that the discrete model successfully captures the irreversibility of certain abstracted physical states, forcing the planner to evaluate the long-term consequences of its thermal manipulations.

3 The Continuous PDDL+ Model

While the discrete model successfully validates the sequential logic of the robot, it lacks the expressiveness required to simulate real-world physics. To overcome the granularity problem of instantaneous thermal jumps and to introduce environmental autonomy, the domain was upgraded to the PDDL+ paradigm. This transition incorporates continuous time ($\#t$), autonomous events, and differential processes.

3.1 Continuous Time and Differential Processes

In the PDDL+ model, the discrete `heat-step` action was entirely removed. Instead, thermodynamics and fluid dynamics are modeled as continuous processes that evolve strictly over time.

To accurately model real-world thermodynamics without introducing complex existential quantifiers, we utilized the PDDL+ feature of simultaneous continuous effects. The cooling process is defined globally for any non-empty kettle above 20°C , while the heating process is defined specifically for a kettle placed on an active heater. By assigning a heating derivative of $+2.5^\circ\text{C}/s$, the planner dynamically calculates the net thermal derivative based on the kettle's state through algebraic summation:

$$\text{Off Heater: } \frac{d(\text{temperature}_k)}{dt} = -0.5 \left[\frac{^\circ\text{C}}{s} \right] \quad \Bigg| \quad \text{On Heater: } \frac{d(\text{temperature}_k)}{dt} = +2.5 - 0.5 = +2.0 \left[\frac{^\circ\text{C}}{s} \right]$$

The water filling mechanism is governed by its own independent linear equation:

$$\frac{d(\text{water-volume}_k)}{dt} = 100.0 \left[\frac{\text{ml}}{s} \right]$$

Furthermore, once the hot water is poured into the cup, it initiates its own independent cooling process to lower the temperature without any active intervention from the robot:

$$\frac{d(\text{temperature}_c)}{dt} = -1.0 \left[\frac{^\circ\text{C}}{\text{s}} \right]$$

3.2 Autonomous Safety Events

A major limitation of the discrete model is the inability of the environment to react independently. In the PDDL+ domain, we introduced instantaneous `:event` blocks to simulate physical safety limits. These events monitor the continuous variables and trigger automatically when thresholds are reached, regardless of the robot's current manipulation task.

Two critical safety events were implemented:

- **stop-filling-safety:** Prevents overflowing by automatically turning off the dispenser and capping the volume when the water volume reaches exactly 1000.0 ml.
- **overheat-safety-switch-off:** Simulates a thermal fuse that instantly deactivates the heater if the water reaches the boiling point (100.0 °C).

3.3 The Robot Cooldown Mechanism

Introducing continuous background processes creates a known heuristic vulnerability in temporal planners: the solver might exploit the continuous time by accumulating resources over long waiting periods, subsequently executing an unrealistic number of instantaneous actions at a single timestamp t .

To enforce physical realism and simulate mechanical inertia, we engineered a `robot-cooldown` pattern. A numeric fluent acts as a timer: every physical manipulation action (e.g., picking, placing, pouring) sets this fluent to 1.0. A continuous background process then decreases it at a rate of 1.0/s. Since every physical action requires the precondition (`<= (robot-cooldown) 0.0`), the robot is strictly throttled to a realistic execution rate, effectively preventing action-stacking exploits and forcing the planner to optimize its timeline realistically.

3.4 Test Case 3: Autonomous Safety and Thermal Tracking

To explicitly demonstrate the robustness of the autonomous `:event` blocks and the process interactions, we designed a complex emergency scenario. At $t = 0$, Kettle 1 (`k1`) is actively filling and will overflow in 1.0 second. Kettle 2 (`k2`) is heating and will boil in exactly 0.5 seconds. The robot's hands are empty, meaning it technically has the capability to intervene. The goal is to prepare flavored water and warm tea.

```
0.0: -----waiting----- [162.0]
162.0: (pick-from-dispenser k1 d1)
162.0: -----waiting----- [163.0]
163.0: (pour-water k1 c-water)
164.0: (place-on-table k1)
165.0: (turn-on-heater h1 k2)
165.0: -----waiting----- [241.0]
241.0: (pick-from-heater k2 h1)
241.0: -----waiting----- [276.0]
276.0: (pour-water k2 c-tea)
276.0: -----waiting----- [281.0]
281.0: (place-on-dispenser k2 d1)
281.0: -----waiting----- [282.0]
282.0: (pick-ingredient drops1)
282.0: -----waiting----- [283.0]
283.0: (prepare-flavored-water)
283.0: -----waiting----- [284.0]
284.0: (pick-ingredient teal)
284.0: -----waiting----- [285.0]
285.0: (prepare-warm-tea)
```

The solver generated a plan that exploits both the environment's autonomy and the global thermal processes:

- **Heuristic Trust (0.0 - 1.0):** Despite having empty hands, the robot executes no actions. Behind the scenes, `k2` hits 100°C at 0.5s, and `k1` hits 1000ml at 1.0s. The `:event` blocks fire autonomously, safely shutting down the appliances. The planner optimizes its action count by letting the domotic safety systems handle the emergencies.
- **Event as a Timer (165.0 - 241.0):** Later, the robot reignites the heater under `k2`. Notably, it never executes a `turn-off` action. At exactly 205.0s, the water reaches 100°C and the safety event fires again, acting as an automatic shut-down timer.
- **Thermal Precision (241.0+):** The robot lets the water dynamically cool down off the active heater. It pours the liquid at 64.5°C and waits for the cup to cool down to 55.5°C before preparing the tea, perfectly satisfying the $55 < T \leq 65^\circ\text{C}$ range constraint.

4 Reflection and Limitations

The transition from a discrete representation to a continuous PDDL+ model successfully bridged the gap between abstract logical planning and physical simulation. The most rewarding design choice was the implementation of Process Interactions for thermodynamics. By defining a persistent `cooling` process alongside an exclusive `heating` process, the solver natively calculated the net thermal derivatives without complex existential quantifiers.

4.1 Limitations and Planner Behavior

Despite the mathematical accuracy of the model, testing with ENHSP revealed specific limitations inherent to heuristic continuous planning.

First, the heuristic struggles with temporal makespan optimization. As seen in Test Case 3, the planner generates functionally valid but practically inefficient wait times (e.g., waiting 160 seconds for boiling water to passively cool rather than taking immediate action). This highlights the lack of energy and time-optimization metrics in standard satisficing planners.

Secondly, real-world thermodynamics dictates that cooling rates are not linear, where dissipation accelerates at higher temperatures. Our model utilizes linear derivatives. Implementing non-linear differential equations is currently beyond the scope of robust planning without severely degrading solver performance.

In conclusion, while the domain accurately handles continuous dynamics, event-driven autonomy, and precise temporal constraints, true optimization in robotic environments would require multi-objective solvers capable of minimizing total makespan alongside logical satisfiability.